

Embedded Systems Authentication and Encryption Using Strong PUF Modeling

Md Shahed Enamul Quadir and John A. Chandy

Electrical and Computer Engineering

University of Connecticut

Storrs, CT 06269

Email: {md.enamul_quadir, john.chandy}@uconn.edu

Abstract—Consumer electronics and embedded systems must ensure trusted authentication and secure communications. Communication session encryption keys are negotiated in a key agreement protocol enabled by a public key/private key pair for authentication. However, embedded systems are limited in their capability to implement public key encryption and client-side authentication. In this paper, we introduce an authentication and session key generation mechanism using Physical Unclonable Functions (PUFs) that extracts randomness from device physical characteristics. The method uses PUF models to help with key exchange and creates periodically refreshable. The approach mitigates tampering attacks as no key is stored on device.

Index Terms—session key, Physical Unclonable Functions (PUFs), key generation, dynamic key

I. INTRODUCTION

Networks of consumer electronics and embedded systems are involved in most parts of our daily life. Therefore, it is essential that these systems ensure data confidentiality, integrity and privacy as well as provide mechanisms to trust device identity and authenticity. The authenticity of the electronic devices is an important factor because it confirms and strengthens the safety of the home electronics network and gives access to the user to control the network securely. [1]. However, most consumer home electronics services rarely have mechanisms for client-side authentication such as certificates or public keys. As a result, communications could be attacked in a number of ways - e.g. man-in-the-middle attack, denial of service attacks, static key leakage, etc. Thus, the credibility and authenticity during communications need to be guaranteed by the consumer devices which are connected to the home network. Without strict client authentication, an unauthorized device could try to access a consumer electronics network with sensors, alarms, smartwatch, IoT, etc. for malicious purposes. For example, in a smart home system, a malicious smoke sensor could insert false messages into a home security control network and set off the alarm and automated activation of water from a sprinkler system. Furthermore, in addition to authentication, secure and confidential communication between devices is crucial to ensure privacy. Security and privacy vulnerabilities issues from these devices could be severe as an attacker could collect sensitive data and information from the network system [2], [3].

Consumer electronic networks have found their way into a wide range of applications domains [2]. Example devices in

these application areas include medical (pacemakers, heart-rate monitors, and insulin pumps), home (sensor, alarm, Google Home, smart thermostat), and personal (smartphones, smartwatch, portable music players, tablets, and laptops). (See Fig 1). In this paper, we have focused on consumer electronics networks that need authenticated communication with a central server. Fig. 2 shows a consumer home security network with the proposed authentication framework. The server manages the consumer devices - for example, a home automation controller managing sensors, alarms, cellular phones, smart TVs, IoT, etc. Those devices need to communicate with the server with secure encryption.

A. Transport Layer Security (TLS)

An obvious choice to provide security for consumer electronics and embedded systems communications is the Transport Layer Security (TLS) protocol which forms the backbone of most secure Internet communications. TLS is a standard encryption protocol that provides privacy and security between communicating users during web browsing, email delivery, instant messaging, and voice over IP (VoIP) [4]. TLS guarantees data confidentiality and authenticity between the server and client during communication by ensuring that no third party or eavesdropper can eavesdrop on or fake a message. The TLS protocol starts with a handshaking procedure by using an asymmetric cipher to establish the secure connection. Public-key based certificates are used for the authentication of the server and the communicating client. Without authentication, an unauthorized client could easily impersonating a legitimate client. During the initial handshaking, TLS establishes a symmetric encryption key for the communication session using public keys or Diffie-Hellman key exchange. After this initial handshaking, the communication is encrypted with the session specific symmetric key. However, if the attacker is able to steal this session key, they could extract the data and potentially all prior data communications. The connection will continue communication with the current key for encryption and decryption until the session ends.

Although TLS enables computer systems to establish secure connections with session keys, TLS is not practical for embedded systems. Low-end processors in most embedded systems do not have the computation capabilities to efficiently implement the public key cryptography necessary for TLS

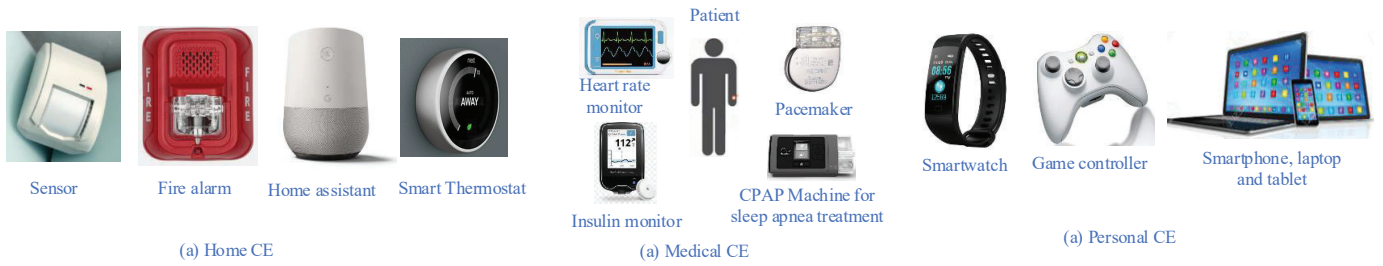


Fig. 1. Examples of consumer electronic (CE) products which are classified as (a) home CE, (b) medical CE and (c) personal CE.

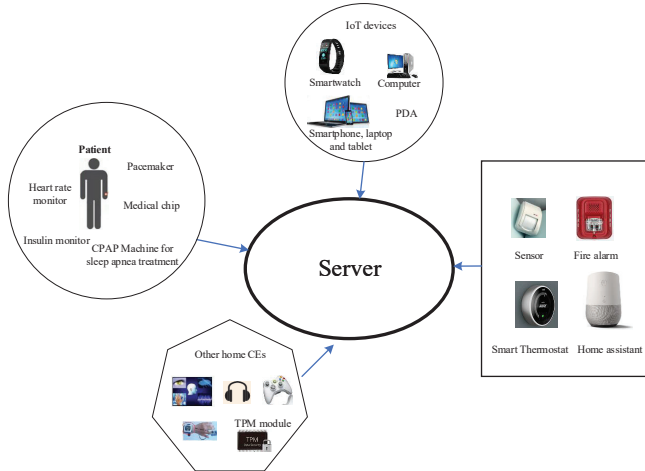


Fig. 2. A platform of home electronics communication network with authentication framework.

authentication or the Diffie-Hellman key exchange protocol. Moreover, even if public key cryptography was computationally feasible, typically only server-side authentication can be implemented. Client-side authentication requires the deployment of private keys in the embedded system, making the system vulnerable to memory probing that can recover these stored private keys [5]–[9]. If the key is static and shared among all devices, breaking the key will allow fake devices to present themselves as authentic. Moreover, if the key is used to establish the session key, a broken key could expose all past and future communication. Therefore, recent versions of the TLS protocol have recommended the use of ephemeral keys to ensure forward secrecy. To communicate with each other, a session specific symmetric key will ensure the encryption of all messages for only a particular time frame. Furthermore, forward secrecy will protect past communications or session against future attacks of the secret key. If a unique session key is generated for each user or client, one compromising session will not impact other sessions data communication other than for this session which was encrypted by that specific key.

B. Physical Unclonable Functions

One possible mechanism to generate private keys without storing the keys is to use physical unclonable functions

(PUFs), a promising security primitive that can be used for secret key storage, identification, authentication and cryptography [10]–[13] (See Fig. 3). PUFs depend on manufacturing variations to present a unique fingerprint (or set of fingerprints) in response to a set of challenge inputs. PUFs can be categorized as strong PUFs or weak PUFs (also called Physical Obfuscated Keys or POKs [14]). The distinguishing feature is that strong PUFs have a large enough challenge space such that enumerating all the challenges is infeasible while weak PUFs or POKs have just one possible challenge or limited set of challenges. Strong PUFs are typically used for authentication, while POKs can be used to generate private keys. However, current weak PUF based key generation methods are static which means the keys do not change over time, and thus is not suitable as an ephemeral session key, and the limited challenge/response pair space restricts its authentication capability. In this work, we propose a unique session key generation method for embedded devices by creating a strong PUF circuit model that can be evaluated by the server. Furthermore, the client or device can be authenticated as well because the client/device specific session keys will work only for specific PUF devices which have been parameterized and have a saved machine learning model in the server site.

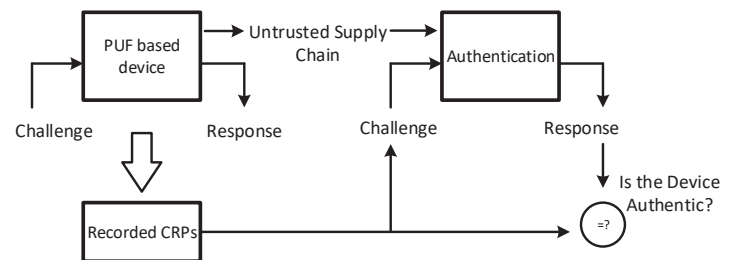


Fig. 3. Secure key generation of devices using physical unclonable functions (PUFs).

II. PROPOSED APPROACH

As mentioned above, POKs have been used for key generation but can not be used to generate ephemeral session keys. In this paper, we propose a session key generation method using strong PUFs without the requirement of secure memory, expensive tamper-resistant hardware, or a PUF access channel.

The core of the approach is twofold: (1) Use a model of the PUF to simulate the behavior of the PUF without having direct access to PUF channels (2) Refresh keys periodically using a response challenge feedback loop.

As with most PUF protocols, we have two phases: *enrollment*, when the PUF model is constructed, and *key reconstruction*, when the PUF is activated to create session keys.

A. Enrollment

In the proposed method, the device will be provided with a unique serial number (See Fig. 4). Before the device is distributed to end users, the PUF will be characterized through open channels that allow access to the PUF input and output. Machine learning algorithms have been used to attack strong PUFs in order to develop a mathematical model of the PUF [15]–[17]. In this work, we use these same type of algorithms to develop a model that, instead of being used to attack the PUF, will be used by a server to simulate the PUF. The parameters/features of the model PUF (PUF_M) are derived and stored in the server. Each database entry in the server consists of a device serial number along with associated PUF model parameters. The space required to store all this data will be still smaller than saving many challenge/response pairs in the server because of the linear number of components. After the PUF model has been characterized, any open channels providing access to the PUF will be removed or fused out.

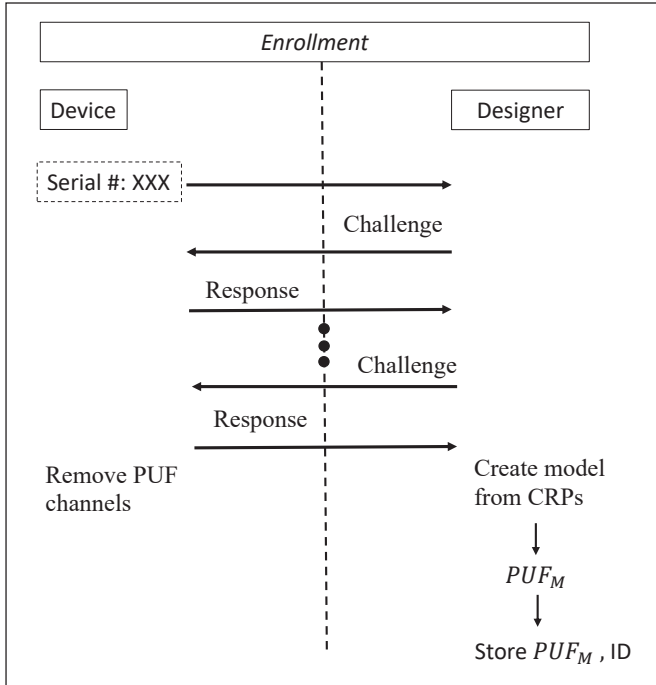


Fig. 4. Enrollment phase for session key generation.

B. Key Reconstruction

Before starting a communication session, both the device and server must reconstruct a common key using the PUF and

simulated PUF respectively. The device first presents the server with its unique serial number, and the server will retrieve the database entry associated with the provided serial number. The first time that a key is created, the device internally presents the serial number as a challenge to the device PUF. The response can be used to generate the session key through some function G , where G could be the identity function or a hash function if needed to generate an appropriate length key with uniform distribution. The server can retrieve the PUF model PUF_M associated with the device's serial ID and recreate the same session key using the simulated PUF model (See Fig. 5).

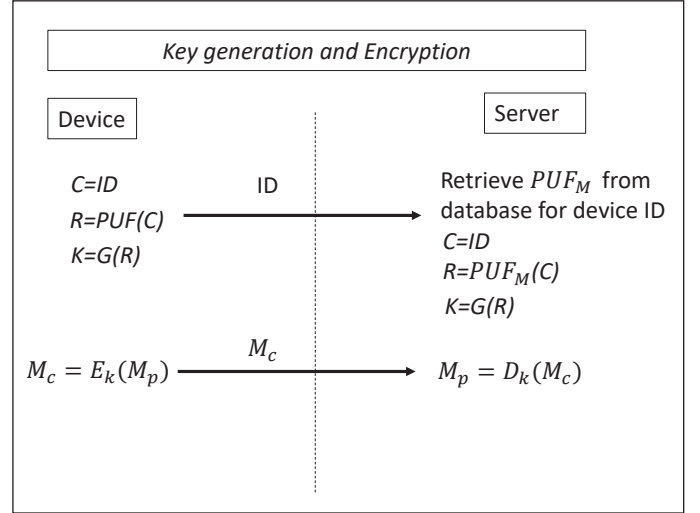


Fig. 5. Session key generation protocol and sending of ciphertext M_C which is the encryption of plaintext M_P using key K derived from PUF or model.

C. Key Refresh

The key reconstruction mechanism, as described, will only allow the creation of a single static key. However, in order to enable ephemeral keys, we need a mechanism to change the keys over time. Previous work has introduced the notion of reconfigurable PUFs which change a strong PUF's functional behavior either logically or physically [18]–[21]. However, these approaches require challenges to be applied to an external PUF interface, thus making the challenges (and responses) visible to potential adversaries who can use this information to generate their own model of the PUF. In our approach, as mentioned above, we remove the PUF access channels so adversaries are not able to apply challenges directly in order to model the PUF. In order to refresh the keys within a device, new keys are generated using a dynamic key generation process as shown in Fig. 6. As before, the key is created from the response of the PUF using the function G . The host server can determine when to refresh the key or the refresh can be triggered by an internal counter or timer. When a key refresh is enabled, the last PUF response is fed back to the input of the PUF through a function F to dynamically update the challenge. The new challenge will generate a new PUF

response which will create an updated key that can be used for new session encryption. The refresh mechanism is defined with the following equations:

$$C_i \leftarrow F(R_{i-1}) \quad (1)$$

$$R_i \leftarrow PUF(C_i) \quad (2)$$

$$K_i \leftarrow G(R_i) \quad (3)$$

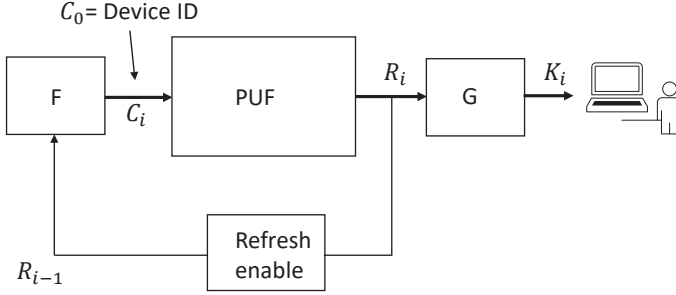


Fig. 6. Block diagram of proposed dynamic key generation.

The server side will follow a similar process to refresh its key so that it corresponds to the key on the client side. Since both the server and client can deterministically arrive at the same key, there is no need for a Diffie-Hellman or public key based key exchange protocol.

D. Error Correction

The PUF output needs to be stable under various supply voltage and temperature conditions [22]. The output value of the PUF device could vary due to several contributing factors including changes in supply voltage or temperature conditions. These variations could alter the output parameters of the PUF devices such as threshold voltages, leakage current, gate voltage, delay, timing and effectively change the output bits of the PUF. In spite of these external variations, the PUF output still needs to remain stable.

In addition, the PUF model may have slight inaccuracies that may cause certain bits in the PUF output to be incorrectly predicted. Thus, both the PUF and model responses, R'_P and R'_M , may have errors. The presence of noise for each key means that the keys need to be corrected in both the device and server side with the use of helper data (Figures 7 and 8). There are a number of error correcting codes that could be used for this purpose, such as Hamming, BCH or Reed Solomon. The resultant helper data from the code will need to be stored in the server database, and provided to the device at initialization and whenever the session key is refreshed. Depending on how much helper data can be stored, that will limit the number of times the session key can be refreshed before the process starts again. The cycle can be extended by using a nonce as an input to the G function as shown in Fig. 7.

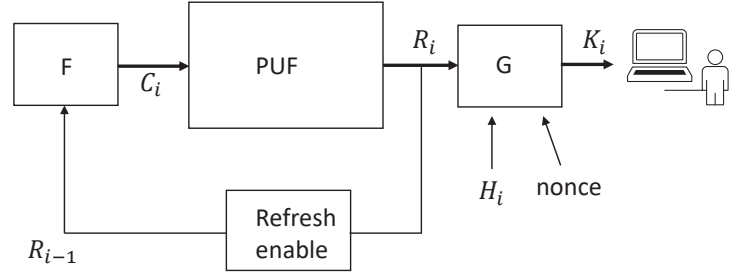


Fig. 7. Block diagram of proposed dynamic key generation with error correction

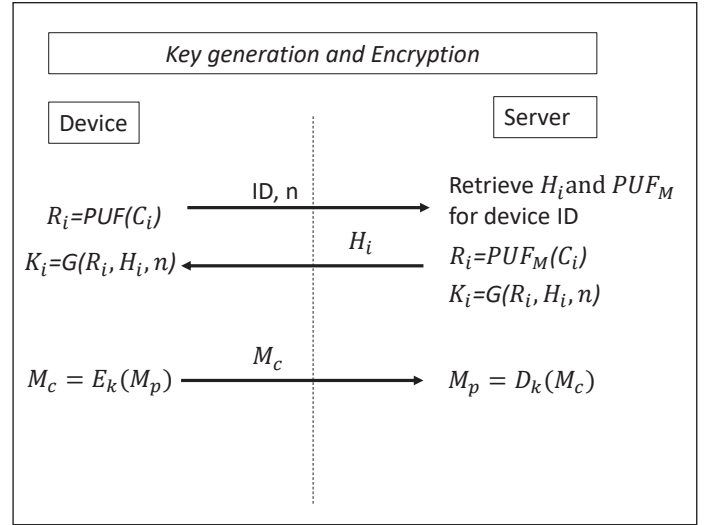


Fig. 8. Session key generation protocol with error correction.

III. DISCUSSION

The proposed protocol removes the need for a public key or certificate based authentication and instead uses a PUF-based identity. The system depends on the fact that the server keeps a database of all possible PUF models. This database is a potential vulnerability point as it exposes the equivalent of a private key for all possible client devices in a particular ecosystem. Since the database is potentially high-value, an attacker may be willing to dedicate significant resources to use aggressive invasive methods to probe the server. Thus, the server may need exceptional anti-tamper features. In a practical system, where a home concentrator device in a security system may be the "server", the concentrator may not keep the database itself, but may instead communicate with a dedicated cloud service to retrieve PUF models. In such a case, the database is less vulnerable, but individual client device PUF data may be exposed within the local server. If an attacker were to even retrieve a single device PUF model, they could use that to impersonate a real device. However, the combination of the refreshing keys and the nonce, would

make it difficult for an attacker to keep in sync with the real device. Moreover, one could use monitoring mechanisms to detect if there are multiple devices with the same ID within the ecosystem.

The approach is similar to that used by the SlenderPUF protocol for device authentication [23] or the Public PUF (PPUF) public key cryptography mechanism [24]. Both SlenderPUF and PPUF use PUF models at the server side to help authenticate a client device. However, SlenderPUF does not address the generation of session keys. PPUF is most similar to our approach and they present a simple key exchange protocol that can be used to generate session keys. PPUFs depend on large timing differences between the simulation and actual PUF to distinguish between private (actual PUF) and public (simulation PUF) keys. However, it is impossible to guarantee that the simulation models will remain computationally difficult as compute resources become cheaper and more powerful. Instead, in our approach, we assume that both the actual and model PUFs are secure and are not shared.

We have, to this point, only discussed client device side authentication. Since we assume that only the server would have access to the device PUF model, if it can establish a common key and is able to decrypt messages, we can be assured that it is an authentic server. If the device PUF model leaks, then this assumption is no longer true.

The devices do not store any secure state in non-volatile memory or fuses that could be probed by an adversary. However, an attacker could potentially decapsulate a device and probe internal buses to activate the PUF and retrieve challenge response pairs in order to construct a PUF model. Such techniques are extremely difficult to do while keeping the device powered and we assume that this is beyond the capability of most adversaries or not worth the effort. As mentioned above, even if an attacker were able to model a single device's PUF, it could not be replicated at scale because monitoring mechanisms should be able to detect that the same ID is being used by copied devices.

If the helper data can not correct errors in the hardware or model PUF, then the server and client will construct different keys and will not be able to communicate. As a result, we assume the error correction is strong enough to correct any errors in the PUF or model with sufficiently high probability.

IV. CONCLUSIONS

As the consumer electronic (CE) market has grown with smart embedded devices, we have addressed symmetric key generation for encryption of the messages of those electronics devices using a low-cost method for customer network frameworks. Our method not only encrypts the data with an ephemeral key but also is able to authenticate the device (client) to a centralized server within a consumer electronic network. In this paper, we have proposed a dynamic symmetric session key generation method using strong PUFs and a server-side simulated model of the PUF. With our proposed method, consumer electronic devices will be able to communicate in the network with dynamic session keys which will make the

message and information secure and trustworthy. The proposed PUF based session key-based encryption will provide a trustworthy embedded system communication platform which will provide a secure exchange and access of sensitive data. The market of consumer embedded electronic devices will continue to expand; therefore, vendors will need a low-cost secure technique for data encryption as well as authentication of those devices. Trustworthiness of the consumer electronic product requires secure data communication as well as the prevention of any loss of credentials and information of that device. Our proposed session key solution will help to secure the encrypted data during the communication in a lightweight and easily adaptable hardware framework which could be suitable for any form of consumer devices.

REFERENCES

- [1] Y.-k. Lee, J.-w. Han, and K.-i. Chung, "Home device authentication method in ubiquitous environment," in *2007 IEEE International Symposium on Consumer Electronics*. IEEE, 2007, pp. 1–5.
- [2] D. A. Hahn, A. Munir, and S. P. Mohanty, "Security and privacy issues in contemporary consumer electronics [energy and security]," *IEEE Consumer Electronics Magazine*, vol. 8, no. 1, pp. 95–99, 2018.
- [3] G. Neven, G. Baldini, J. Camenisch, and R. Neisse, "Privacy-preserving attribute-based credentials in cooperative intelligent transport systems," in *2017 IEEE Vehicular Networking Conference (VNC)*. IEEE, 2017, pp. 131–138.
- [4] T. Dierks and E. Rescorla, "The transport layer security (TLS) protocol version 1.2," Network Working Group, Tech. Rep. RFC 5246, August 2008.
- [5] C. DeNardi, R. Desplats, P. Perdu, J. Gauffier, and C. Guérin, "De-scrambling and data reading techniques for flash-EEPROM memories. application to smart cards," *Microelectronics Reliability*, vol. 46, no. 9, pp. 1569–1574, 2006.
- [6] C. De Nardi, R. Desplats, P. Perdu, C. Guérin, J. L. Gauffier, and T. B. Amundsen, "Direct measurements of charge in floating gate transistor channels of flash memories using scanning capacitance microscopy," in *International Symposium for Testing and Failure Analysis*, vol. 32. ASM International; 1998, 2006, p. 86.
- [7] C. De Nardi, R. Desplats, P. Perdu, F. Beaudoin, and J.-L. Gauffier, "Oxide charge measurements in EEPROM devices," *Microelectronics Reliability*, vol. 45, no. 9, pp. 1514–1519, 2005.
- [8] J.-M. Cioranescu, J.-L. Danger, T. Graba, S. Guilley, Y. Mathieu, D. Naccache, and X. T. Ngo, "Cryptographically secure shields," in *Hardware-Oriented Security and Trust (HOST)*, 2014 IEEE International Symposium on. IEEE, 2014, pp. 25–31.
- [9] S. E. Quadir, J. Chen, D. Forte, N. Asadizanjani, S. Shahbazmohamadi, L. Wang, J. Chandy, and M. Tehranipoor, "A survey on chip to system reverse engineering," *ACM journal on emerging technologies in computing systems (JETC)*, vol. 13, no. 1, p. 6, 2016.
- [10] G. E. Suh and S. Devadas, "Physical unclonable functions for device authentication and secret key generation," in *2007 44th ACM/IEEE Design Automation Conference*. IEEE, 2007, pp. 9–14.
- [11] A. Maiti and P. Schaumont, "Improving the quality of a physical unclonable function using configurable ring oscillators," in *2009 International Conference on Field Programmable Logic and Applications*. IEEE, 2009, pp. 703–707.
- [12] M. Majzoobi, F. Koushanfar, and M. Potkonjak, "Techniques for design and implementation of secure reconfigurable PUFs," *ACM Transactions on Reconfigurable Technology and Systems (TRETS)*, vol. 2, no. 1, p. 5, 2009.
- [13] J. Zhang, L. Wan, Q. Wu, and G. Qu, "DMOS-PUF: Dynamic multi-key-selection obfuscation for strong PUFs against machine learning attacks," *arXiv preprint arXiv:1806.02011*, 2018.
- [14] R. S. Khan, N. Kanan, C. Jin, J. Scoggin, N. Noor, S. Muneer, F. Dirisaglik, P. H. Nguyen, H. Silva, M. van Dijk *et al.*, "Intrinsically reliable and lightweight physical obfuscated keys," *arXiv preprint arXiv:1703.07427*, 2017.

- [15] U. Rührmair, F. Sehnke, J. Sölter, G. Dror, S. Devadas, and J. Schmidhuber, "Modeling attacks on physical unclonable functions," in *Proceedings of the 17th ACM conference on Computer and communications security*. ACM, 2010, pp. 237–249.
- [16] D. P. Sahoo, P. H. Nguyen, D. Mukhopadhyay, and R. S. Chakraborty, "A case of lightweight PUF constructions: Cryptanalysis and machine learning attacks," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 34, no. 8, pp. 1334–1343, 2015.
- [17] U. Rührmair, J. Sölter, F. Sehnke, X. Xu, A. Mahmoud, V. Stoyanova, G. Dror, J. Schmidhuber, W. Burleson, and S. Devadas, "PUF modeling attacks on simulated and silicon data," *IEEE Transactions on Information Forensics and Security*, vol. 8, no. 11, pp. 1876–1891, 2013.
- [18] S. Katzenbeisser, Ü. Kocabaş, V. van der Leest, A.-R. Sadeghi, G.-J. Schrijen, and C. Wachsmann, "Recyclable PUFs: Logically reconfigurable PUFs," *Journal of Cryptographic Engineering*, vol. 1, pp. 177–186, 2011.
- [19] K. Kursawe, A. Sadeghi, D. Schellekens, B. Skoric, and P. Tuyls, "Reconfigurable physical unclonable functions - enabling technology for tamper-resistant storage," in *2009 IEEE International Workshop on Hardware-Oriented Security and Trust*, July 2009, pp. 22–29.
- [20] Y. Lao and K. K. Parhi, "Reconfigurable architectures for silicon physical unclonable functions," in *2011 IEEE International Conference on Electro/Information Technology*, May 2011, pp. 1–7.
- [21] I. Eichhorn, P. Koeberl, and V. van der Leest, "Logically reconfigurable PUFs: Memory-based secure key storage," in *Proceedings of the Sixth ACM Workshop on Scalable Trusted Computing*, ser. STC '11. New York, NY, USA: ACM, 2011, pp. 59–64. [Online]. Available: <http://doi.acm.org/10.1145/2046582.2046594>
- [22] V. Vivekraj, "Low-power, stable and secure on-chip identifiers design," Ph.D. dissertation, Virginia Tech, 2010.
- [23] M. Majzoobi, M. Rostami, F. Koushanfar, D. S. Wallach, and S. Devadas, "Slender PUF protocol: A lightweight, robust, and secure authentication by substring matching," in *2012 IEEE Symposium on Security and Privacy Workshops*. IEEE, 2012, pp. 33–44.
- [24] N. Beckmann and M. Potkonjak, "Hardware-based public-key cryptography with public physically unclonable functions," in *Information Hiding*, S. Katzenbeisser and A.-R. Sadeghi, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 206–220.